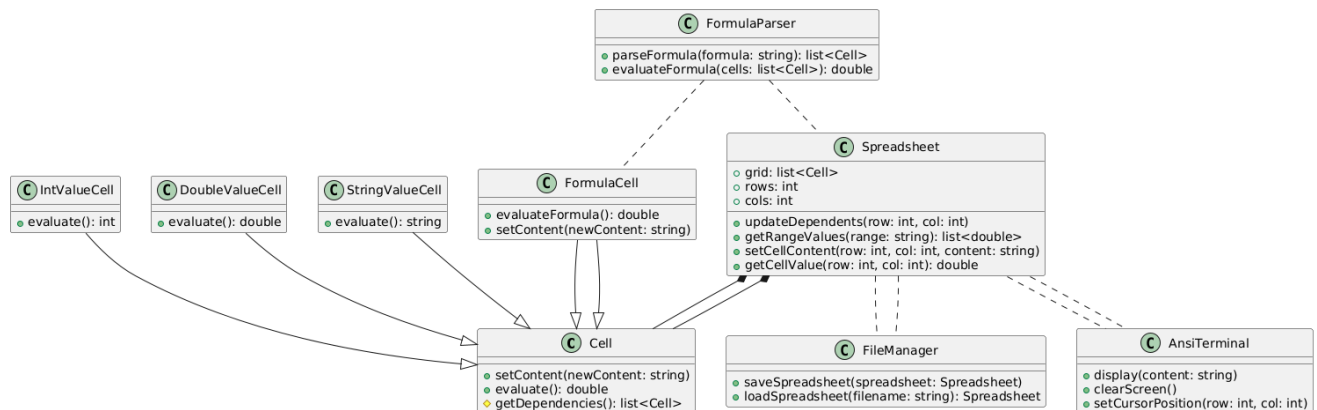


CSE 241 OOP HOMEWORK 2 PRESENTATION DOCUMENT

In this pdf document, I will briefly explain the assignment to you with examples about the purpose of the codes, the uml diagram, the output of the code.

In this assignment, it was an assignment in which we made the classes we created in the previous homework 1 virtual and added new features. Let's start the presentation by showing the UML diagram of this assignment.



As you can see, it contains a very complex UML diagram.

Let's start with the cell class first:

```
1 #ifndef CELL_H
2 #define CELL_H
3
4 #include <string>
5 #include <vector>
6 #include <utility>
7
8 class Spreadsheet; // Forward declaration
9
10 class Cell {
11 public:
12     virtual ~Cell() = default;
13
14     virtual void setContent(const std::string& newContent, Spreadsheet& spreadsheet) = 0; // Yeni imza
15     virtual std::string getContent() const = 0;
16
17     virtual void setValue(double newValue) = 0;
18     virtual double getValue() const = 0;
19
20     virtual bool isFormula() const = 0;
21
22     virtual void addDependent(int row, int col) = 0;
23     virtual std::vector<std::pair<int, int>> getDependents() const = 0;
24     virtual void clearDependents() = 0;
25 };
26
27 #endif // CELL_H
```

This code defines an abstract class (**Cell**) for modelling spreadsheet cells and forms the basis for four different classes (e.g. text cell, number cell, formula cell, etc.).

Header Guard: Prevents the file from being included more than once.

Libraries: Manages cell content (string) and dependencies (vector<pair<int, int>>).

Main Functions:

Content and value management (setContent, getValue, etc.).

Formula control (isFormula).

Handling cell dependencies (add, get, clear).

This structure provides an interface that defines the basic behaviour of cells.

Here are brief descriptions of the classes derived from the Cell class according to the given project requirements:

IntValueCell:

```
2 #define INTVALUECELL_H
3
4 #include "Cell.h"
5 #include <string>
6 #include <vector>
7 #include <utility>
8 #include "Spreadsheet.h"
9
10
11 class IntValueCell : public Cell {
12 private:
13     int value;
14     std::string content;
15     std::vector<std::pair<int, int>> dependencies;
16
17 public:
18     void setContent(const std::string& newContent, Spreadsheet& spreadsheet) override;
19
20     std::string getContent() const override;
21
22     void setValue(double newValue) override;
23     double getValue() const override;
24
25     bool isFormula() const override;
26
27     void addDependent(int row, int col) override;
28     std::vector<std::pair<int, int>> getDependents() const override;
29     void clearDependents() override;
30 };
31
32 #endif // INTVALUECELL_H
```

Objective: A class of cells representing integer values.

Properties: Stores and processes only integer values in the cell.

Behaviour:

The setValue and getValue functions work only with the int type.

Does not contain a formula (the isFormula function returns false).

DoubleValueCell:

```

1 #ifndef DOUBLEVALUECELL_H
2 #define DOUBLEVALUECELL_H
3
4 #include "cell.h"
5 #include <string>
6 #include <vector>
7 #include <utility>
8 #include "Spreadsheet.h"
9
10 class DoubleValueCell : public Cell {
11 private:
12     double value;
13     std::string content;
14     std::vector<std::pair<int, int>> dependents;
15
16 public:
17     void setContent(const std::string& newContent, Spreadsheet& spreadsheet) override;
18     std::string getContent() const override;
19
20     void setValue(double newValue) override;
21     double getValue() const override;
22
23     bool isFormula() const override;
24
25     void addDependent(int row, int col) override;
26     std::vector<std::pair<int, int>> getDependents() const override;
27     void clearDependents() override;
28 };
29
30 #endif // DOUBLEVALUECELL_H

```

Purpose: A class of cells representing floating-point decimal values.

Properties: Stores and manipulates decimal values in the cell.

Behaviour:

The setValue and getValue functions only work with the double type.

Suitable for calculations requiring high accuracy.

Does not contain a formula (isFormula returns false).

StringValueCell:

```

1 #ifndef STRINGVALUECELL_H
2 #define STRINGVALUECELL_H
3
4 #include "Cell.h"
5 #include <string>
6 #include <vector>
7 #include <utility>
8
9 class StringValueCell : public Cell {
10 private:
11     std::string value;
12     std::string content;
13     std::vector<std::pair<int, int>> dependents;
14
15 public:
16     void setContent(const std::string& newContent, Spreadsheet& spreadsheet) override;
17     std::string getContent() const override;
18     void setValue(double newValue) override;
19     double getValue() const override;
20     bool isFormula() const override;
21     void addDependent(int row, int col) override;
22     std::vector<std::pair<int, int>> getDependents() const override;
23     void clearDependents() override;
24 };
25
26 #endif // STRINGVALUECELL_H
27
28

```

Purpose: A class of cells representing text (string) values.

Properties: Stores only text values in the cell.

Behaviour:

The setContent and getContent functions organise the cell content as text.

Does not contain values (doubles), only textual content.

Does not contain a formula (isFormula returns false).

FormulaCell:

```

5 #include "Spreadsheet.h"
6 #include <string>
7 #include <vector>
8 #include <utility>
9 #include "FormulaParser.h"
10
11 class FormulaCell : public Cell {
12 private:
13     double value;
14     std::string content;
15     std::vector<std::pair<int, int>> dependents;
16
17     // Formül çözümü fonksiyonu
18     double evaluateFormula(const std::string& formula, const Spreadsheet& spreadsheet);
19
20 public:
21     void setContent(const std::string& newContent, Spreadsheet& spreadsheet) override;
22     std::string getContent() const override;
23     void setValue(double newValue) override;
24     double getValue() const override;
25     bool isFormula() const override;
26
27     void addDependent(int row, int col) override;
28     std::vector<std::pair<int, int>> getDependents() const override;
29     void clearDependents() override;
30 };
31
32 #endif // FORMULACELL_H
33

```

Purpose: A class of cells representing formulae and calculations.

Properties: Stores and evaluates a formula (e.g. =A1 + B2) in a cell.

Behaviour:

Works with the formula parser.

The setContent function gets the formula as text.

The getValue function returns the result of the formula.

Manages dependents, because formulas can reference other cells.

isFormula returns true.

These classes extend the interface provided by the Cell class to support different data types and behaviours. This structure provides the basis for a flexible spreadsheet implementation in the project.

FormulaParser:

```
1 #ifndef FORMULAPARSER_H
2 #define FORMULAPARSER_H
3
4 #include <string>
5 #include "Spreadsheet.h"
6
7 class FormulaParser {
8 public:
9     // Formülü değerlendir ve sonucu döndür
10     static double evaluate(const std::string& formula, const Spreadsheet& sheet);
11 };
12
13 #endif // FORMULAPARSER_H
14
```

This script calculates and returns the result of a formula in a cell.

Analyse Formula: Parses cell references (e.g. A1) and numbers within the expression by removing the = sign.

Calculation: Applies basic mathematical operations (+, -, *, /). Gets the values of cell references from Spreadsheet.

Error Checking: Throws errors for division by zero and invalid formula entries.

Result: Returns the result of the operation as a double.

Example: =A1+3.5 -> takes the value A1, adds it to 3.5, returns the result.

Spreadsheet:

```

11 class Spreadsheet {
12 private:
13     std::vector<std::vector<std::unique_ptr<Cell>>> grid; // 2D hücre tablosu
14     int rows; // Satır sayısı
15     int cols; // Sütun sayısı
16     int selectedRow = 0; // Varsayılan olarak ilk satır
17     int selectedCol = 0; // Varsayılan olarak ilk sütun
18
19     void updateDependents(int row, int col); // Bağımlı hücreleri yeniden hesapla
20     std::vector<std::pair<int, int>> parseFormulaDependencies(const std::string& formula); // Formülden bağımlılıkları çözümle
21
22     // Yeni metod: Hücre türünü belirle ve oluştur
23     std::unique_ptr<Cell> createCell(const std::string& content);
24
25 public:
26     int getSelectedRow() const { return selectedRow; }
27     int getSelectedCol() const { return selectedCol; }
28     void moveSelection(char direction); // Yön hareket metodu
29     double getCellValue(int row, int col) const;
30
31     Spreadsheet(int rowCount, int colCount); // Yapıcı
32
33     void setCellContent(int row, int col, const std::string& content); // Hücre içeriğini ayarla
34     std::string getCellContent(int row, int col) const; // Hücre içeriğini al
35
36     void printSpreadsheet(AnsiTerminal& terminal) const;
37 };
38
39 #endif // SPREADSHEET_H
40

```

This script implements the Spreadsheet class in the spreadsheet application.

Main Functions:

Table Creation: Initialises a dynamic cell structure based on the number of rows and columns.

Cell Type Management: Creates the appropriate cell type (IntValueCell, FormulaCell, etc.) according to the content.

Dependency Management: Tracks the dependencies of formula cells and reflects changes to dependent cells.

User Interaction: Edits the contents of cells, displays the table in the terminal, and manages cell selection.

Formula Analysing: Parses formulas, solves cell references and makes calculations.

Assignment Importance

Dynamic Memory Management: Memory security is provided with smart pointers.

Abstraction: Cell types are derived from the Cell class to create an extensible structure.

Dependency Update: Formulas and dependent cells are updated automatically.

Terminal Interface: User interaction is supported.

This code is an important component that demonstrates the basic logic of the spreadsheet and modern C++ skills.

AnsiTerminal:

Function of Code

AnsiTerminal provides text colouring, positioning and user input management in a terminal-based interface.

Main Functions

Colour Management: Changes and resets text colour and prints coloured text.

Terminal Control: Clears the screen, prints text in a specific position or in reverse colour.

User Input: Detects single key and special key inputs (e.g. directional keys).

Constructor/Destructor: Prepares the terminal settings for input and returns them to their original state.

Importance

The terminal-based spreadsheet provides the basis for visuality (colour and positioning) and interactivity (user input).

FileManager:

```
1 #ifndef FILEMANAGER_H
2 #define FILEMANAGER_H
3
4 #include <string>
5 #include "Spreadsheet.h"
6
7 class FileManager {
8 public:
9     static void saveToCSV(const Spreadsheet& sheet, const std::string& filename);
10    static void loadFromCSV(Spreadsheet& sheet, const std::string& filename);
11 };
12
13 #endif // FILEMANAGER_H
14
```

Function of Code

The FileManager provides the functions of saving and loading the spreadsheet as a CSV file.

saveToCSV: Saves the spreadsheet in CSV format by row and column.

loadFromCSV: Reads the CSV file, loads it into the table and updates the cell contents.

Importance

Provides data persistence, exports and reloads the table. With the CSV format, data becomes portable and easily usable.

Below is an output example, a value was entered into A1 and a value was entered into C1. Then $=A1 + C1$ was written to C2 cell and you can see the output below: You can see some outputs below:

[illegible][illegible][illegible][illegible]

You will now see below how to export this code to a .csv document:

```
=== Main Menu ===
1) Display Spreadsheet
2) Save to CSV
3) Load from CSV
4) Exit
Select an option:
```

While in this section we click on 2

```
=== Main Menu ===
1) Display Spreadsheet
2) Save to CSV
3) Load from CSV
4) Exit
Select an option: Enter filename to save as CSV: 
```

In this section, a file named .csv is written and the table is transferred there.

	Standart	Standart	Standart	Standart
1	12		15	
2			27.000000	
3				
4				
5				

When creating the table from the .csv extension document, the 3rd option is selected and then the .csv file name is written.

Chatgpt-assisted sections:

I got help from Chatgpt ai in FormulaParser.h and FormulaParser.cpp.

Missing Sections:

In this assignment, there are a few deficiencies in Template, namespace and functions.

What I learnt and reinforced in this assignment:

Dynamic Memory and Smart Pointers

Data management with 2D dynamic array without using STL.

Memory management with smart pointers.

Abstract Class and Derivation

Cell abstract class and derivations such as FormulaCell, IntValueCell, StringValueCell, DoubleValueCell.

Namespaces

Logical namespaces for code organisation (Spreadsheet, Utils).

Error handling

C++ exceptions for formula errors, overruns and file operations.

Templates

Use templates for reusable components (Range, Grid).

Formula Processing

Formula parsing and calculation supporting operators and functions.

File Operations

Saving and loading tables in CSV format.

Visual Interface

A text-based, visual and interactive interface with ANSI terminal

Thanks for everything...